

Using TLA+ as a Tool for Building Transactional Memory Models from Specialized Frameworks

Yeremey Pervunetskikh

May 16, 2026

HSE University

There is a connection between the Tsai-Li framework ¹ and Software Transactional Memory (STM) mechanisms

¹J. J. Tsay and H. C. Li, "Lock-free concurrent tree structures for multiprocessor systems (extended abstract)."

Introduction

There is a connection between the Tsai-Li framework ¹ and Software Transactional Memory (STM) mechanisms

First, both use Multiple Compare-And-Swap, where

$$\text{CAS}(\text{value}, \text{old}, \text{new}) \triangleq \vee (\text{value} = \text{old} \wedge \text{value}' = \text{new}) \\ \vee (\text{value} \neq \text{old} \wedge \text{value}' = \text{value}) \text{ atomically}$$

¹J. J. Tsay and H. C. Li, "Lock-free concurrent tree structures for multiprocessor systems (extended abstract)."

Introduction

There is a connection between the Tsai-Li framework ¹ and Software Transactional Memory (STM) mechanisms

First, both use Multiple Compare-And-Swap, where

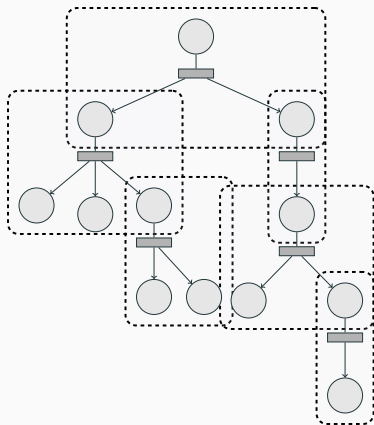
$$\text{CAS}(\text{value}, \text{old}, \text{new}) \triangleq \vee (\text{value} = \text{old} \wedge \text{value}' = \text{new}) \\ \vee (\text{value} \neq \text{old} \wedge \text{value}' = \text{value}) \text{ atomically}$$

Second, we can establish this connection by utilizing TLA+ as a specification tool. For instance, the model can be verified using the TLC model checker

¹J. J. Tsay and H. C. Li, "Lock-free concurrent tree structures for multiprocessor systems (extended abstract)."

Tsay-Li Framework

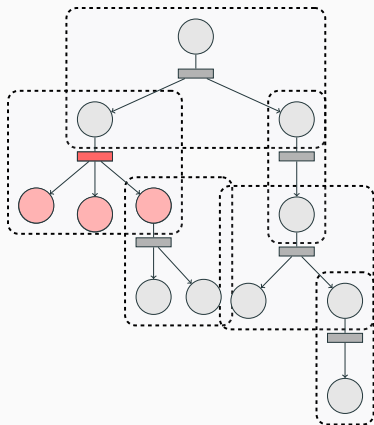
The framework ensures lock-freedom, based on a top-down traversal. It performs frame updates from a changed local copy



Frame structures always overlap

²Natarajan et al. "Concurrent wait-free red black trees."

Tsay-Li Framework



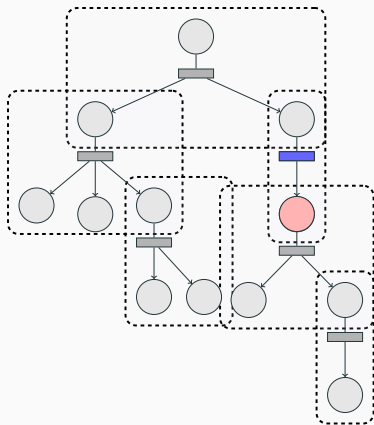
The framework ensures lock-freedom, based on a top-down traversal. It performs frame updates from a changed local copy

Natarajan et. al. modified this approach, subdivided nodes into vertices($\bigcirc$) and subtrees($\square$)²

Frame structures always overlap

²Natarajan et al. "Concurrent wait-free red black trees."

Tsay-Li Framework



The framework ensures lock-freedom, based on a top-down traversal. It performs frame updates from a changed local copy

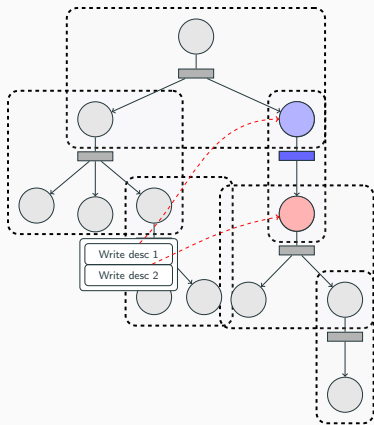
Natarajan et. al. modified this approach, subdivided nodes into vertices($\bigcirc$) and subtrees($\square$)²

A thread may drop out of the tree if a concurrent CAS operation is performed upstream.

Frame structures always overlap

²Natarajan et al. "Concurrent wait-free red black trees."

Tsay-Li Framework



Frame structures always overlap

²Natarajan et al. "Concurrent wait-free red black trees."

The framework ensures lock-freedom, based on a top-down traversal. It performs frame updates from a changed local copy

Natarajan et. al. modified this approach, subdivided nodes into vertices($\bigcirc$) and subtrees($\square$)²

A thread may drop out of the tree if a concurrent CAS operation is performed upstream. Therefore, we need to utilize descriptors and helping approach.

Adaptation

To transition from trees to flat sets, values must be moved from nodes directly into descriptors. It should consider three phases:

- **Acquire:** Thread installs its descriptor D into target cells R_j via CAS in ascending address order (prevents deadlocks). If blocked by descriptor U , the thread helps U finish first.

Adaptation

To transition from trees to flat sets, values must be moved from nodes directly into descriptors. It should consider three phases:

- **Acquire:** Thread installs its descriptor D into target cells R_j via CAS in ascending address order (prevents deadlocks). If blocked by descriptor U , the thread helps U finish first.
- **Decision (Linearization Point):** Once all cells are acquired, the thread atomically changes descriptor status via CAS from *Undecided* to *Committed* or *Aborted*.

Adaptation

To transition from trees to flat sets, values must be moved from nodes directly into descriptors. It should consider three phases:

- **Acquire:** Thread installs its descriptor D into target cells R_j via CAS in ascending address order (prevents deadlocks). If blocked by descriptor U , the thread helps U finish first.
- **Decision (Linearization Point):** Once all cells are acquired, the thread atomically changes descriptor status via CAS from *Undecided* to *Committed* or *Aborted*.
- **Release:** Thread resets cell states to concrete values. Updates to new_j if *Committed*, or reverts to old_j if *Aborted*, exposing the changes to the system.

Adaptation

To transition from trees to flat sets, values must be moved from nodes directly into descriptors. It should consider three phases:

- **Acquire:** Thread installs its descriptor D into target cells R_j via CAS in ascending address order (prevents deadlocks). If blocked by descriptor U , the thread helps U finish first.
- **Decision (Linearization Point):** Once all cells are acquired, the thread atomically changes descriptor status via CAS from *Undecided* to *Committed* or *Aborted*.
- **Release:** Thread resets cell states to concrete values. Updates to new_j if *Committed*, or reverts to old_j if *Aborted*, exposing the changes to the system.

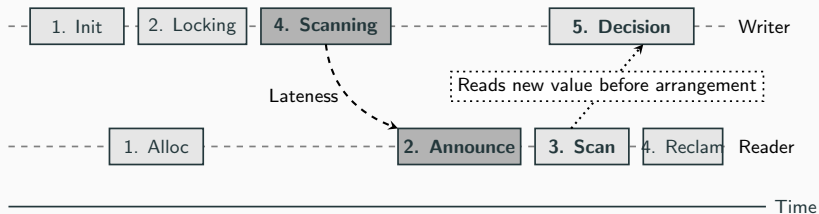
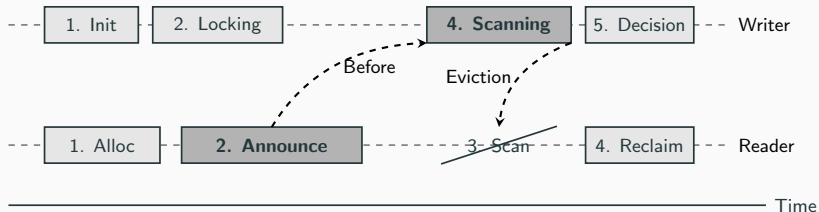
Since the specification draws no difference between threads and their descriptors, all updates occur concurrently.

Algorithm Specifications

State variables:

- `wDescState` Descriptor status (*IDLE*, *ACTIVE*, *SCANNING*, *SUCCESS*, *ABORTED*).
- `wDescPending` Cells awaiting locking or physical update.
- `wDescFrame` Keys (cells) acquired by the transaction.
- `wDescOldValue` Snapshot of original cell values (*old_j*).
- `wDescNewValue` Snapshot of target cell values (*new_j*).

Reading Model



Strategy for linearization point

Formal Methodology & Linearizability Property

- **Variables** `nodeValue` & `nodeDesc` track memory;
 `absNodeValue` is refinement mapping to ideal atomic memory
- **Space Boundary** restricted to `successCount` ≤ 3
- **Liveness** is defined via property similar to TreeCAS module:

$$\begin{aligned} GlobalProgress \triangleq & (\exists d \in WDescs : descState[d] = \text{"ACTIVE"}) \rightsquigarrow \\ & (\exists u \in WDescs : descState[u] = \text{"SUCCESS"}) \\ & \vee \neg StructConstraint \end{aligned}$$

Safety Invariants

- **AtomicityInvariant** synchronizes physical and abstract states. If a cell is free, its value must match the abstract state. If owned by a descriptor, its value depends on the transaction phase
- **ReaderAtomicity** ensures snapshot consistency

$ReaderAtomicity \triangleq \forall r \in RDescriptors : (rDescState[r] = \text{"READING"})$

$\implies \forall n \in rDescFrame[r] :$

$\vee rDescView[r][n] = absNodeValue[n]$

$\vee \exists d \in WDescriptors :$

$\wedge wDescState[d] = \text{"SUCCESS"}$

$\wedge n \in wDescFrame[d]$

$\wedge rDescView[r][n] = wDescOldValue[d][n]$

- **Configuration:** 2 writing threads, 1 reading thread, and a 2-cell memory pool.
- **State Space Constraint:** Increasing the number of threads causes an exponential state-space explosion.

- **Configuration:** 2 writing threads, 1 reading thread, and a 2-cell memory pool.
- **State Space Constraint:** Increasing the number of threads causes an exponential state-space explosion.
- **Sufficiency:** 2 concurrent threads are sufficient because all method phases overlap pairwise. A 3rd thread adds no new interleaving patterns, only duplicating existing overlapping scenarios.
- **Memory Pool:** A 2-cell memory pool is enough to verify how the framework handles overlapping frame sections.

Conclusion

In non-blocking algorithms, the `Help(d, n)` routine is strictly necessary only to guarantee liveness. Disabling it has no effect on the specification without impacting any safety invariants.

Lightweight reading protocol implies linearizability with minimal overhead.

The source code for the specification is available at:



<https://github.com/Yerk08/MCAS-formal-specification/>

Thanks for your attention!